

SPI Communication

Serial Peripheral Interface — ESP32 Complete Guide

Follow F-Code101

01 What is SPI?

SPI (Serial Peripheral Interface) is a **synchronous, full-duplex** communication protocol developed by Motorola. It allows a microcontroller (Master) to talk to one or more peripherals (Slaves) at very high speeds — typically 10–80 MHz — making it the go-to choice for displays, SD cards, sensors, and flash memory.

■	Full-duplex: sends and receives data simultaneously
■	Synchronous: all devices share a clock signal (SCLK)
■	Multiple slaves: each gets its own Chip Select (CS) pin
■	No acknowledgment overhead — very fast transfers
■	ESP32 supports up to 80 MHz SPI clock speed

02 How SPI Works

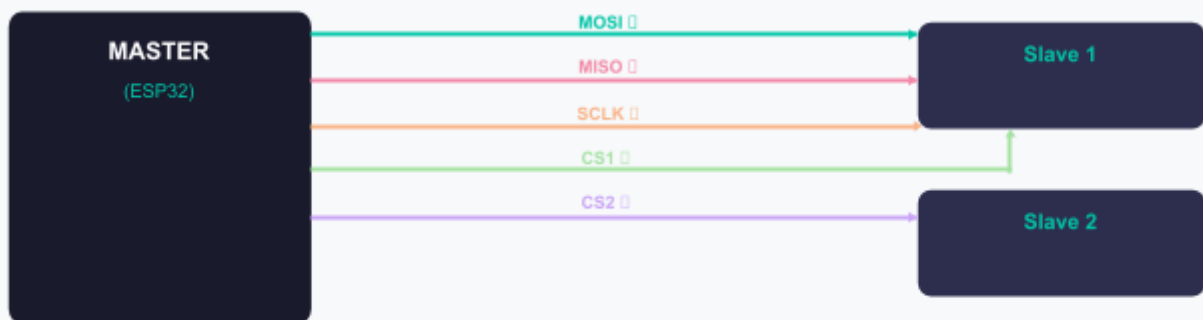
SPI communication uses a **master-slave architecture**. The master controls the clock and initiates every transaction. Data is shifted out on MOSI and shifted in on MISO simultaneously on every clock edge. The CS pin goes LOW to select a slave and returns HIGH when done.

1	Master pulls CS LOW	Selects the target slave device.
2	Master drives SCLK	Clock signal starts toggling all transfers are clock-driven.
3	MOSI shifts data out	Master sends bits MSB-first on each clock pulse.
4	MISO shifts data in	Slave simultaneously sends response bits back to master.
5	CS returns HIGH	Transaction ends; slave is released.

03 The 4 SPI Pins

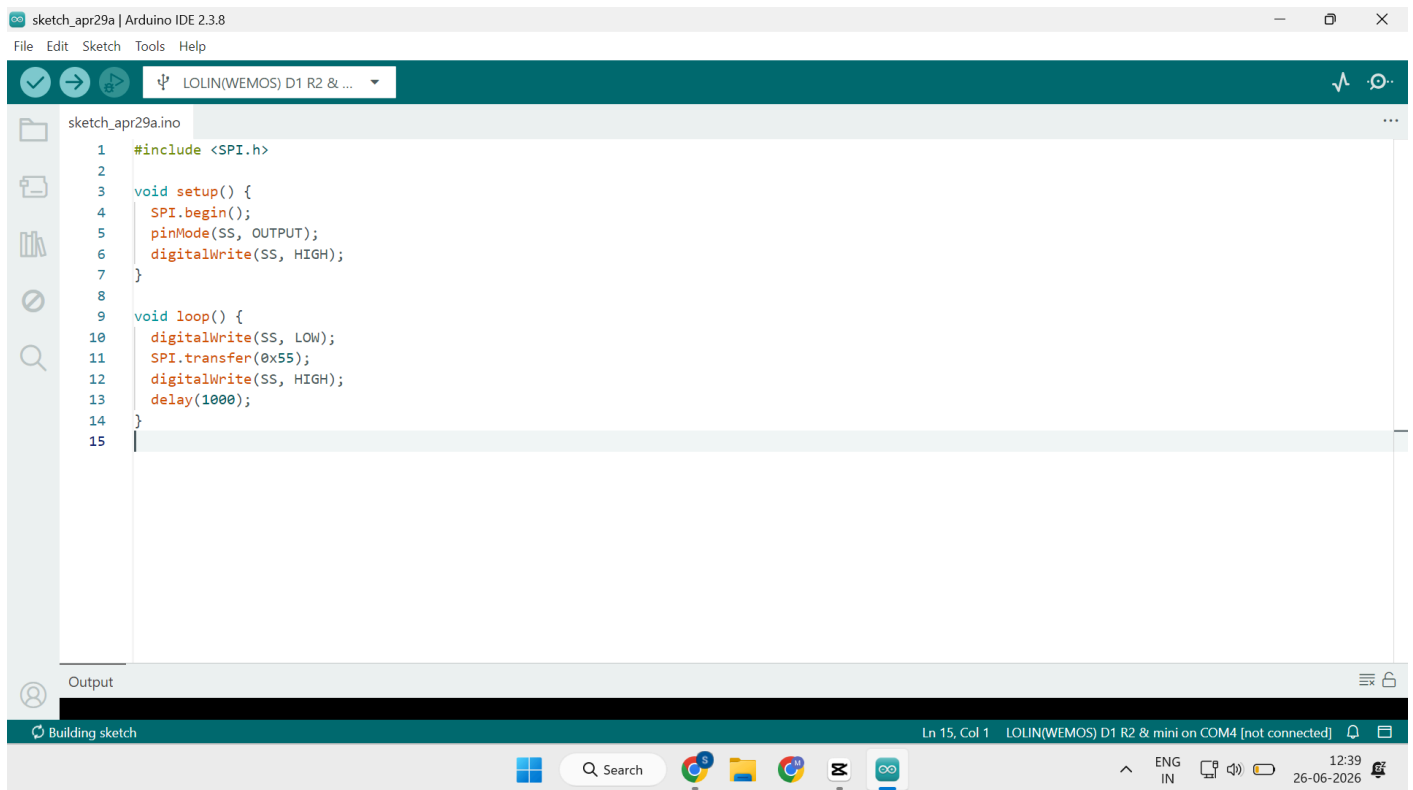
Pin	Full Name	Direction	ESP32 Default	Description
MOSI	Master Out Slave In	Master → Slave	GPIO 23	Data from master to slave
MISO	Master In Slave Out	Slave → Master	GPIO 19	Data from slave to master
SCLK	Serial Clock	Master → Slave	GPIO 18	Clock signal from master
CS	Chip Select (SS)	Master → Slave	GPIO 5	LOW = slave selected

04 Master-Slave Block Diagram



Note: Each Slave gets its own CS line. MOSI, MISO, and SCLK are shared on the same bus. Only one slave is active (CS LOW) at a time

05 ESP32 SPI Master — Arduino Code



```
sketch_apr29a.ino
1  #include <SPI.h>
2
3  void setup() {
4      SPI.begin();
5      pinMode(SS, OUTPUT);
6      digitalWrite(SS, HIGH);
7  }
8
9  void loop() {
10     digitalWrite(SS, LOW);
11     SPI.transfer(0x55);
12     digitalWrite(SS, HIGH);
13     delay(1000);
14 }
15
```

Output

Building sketch

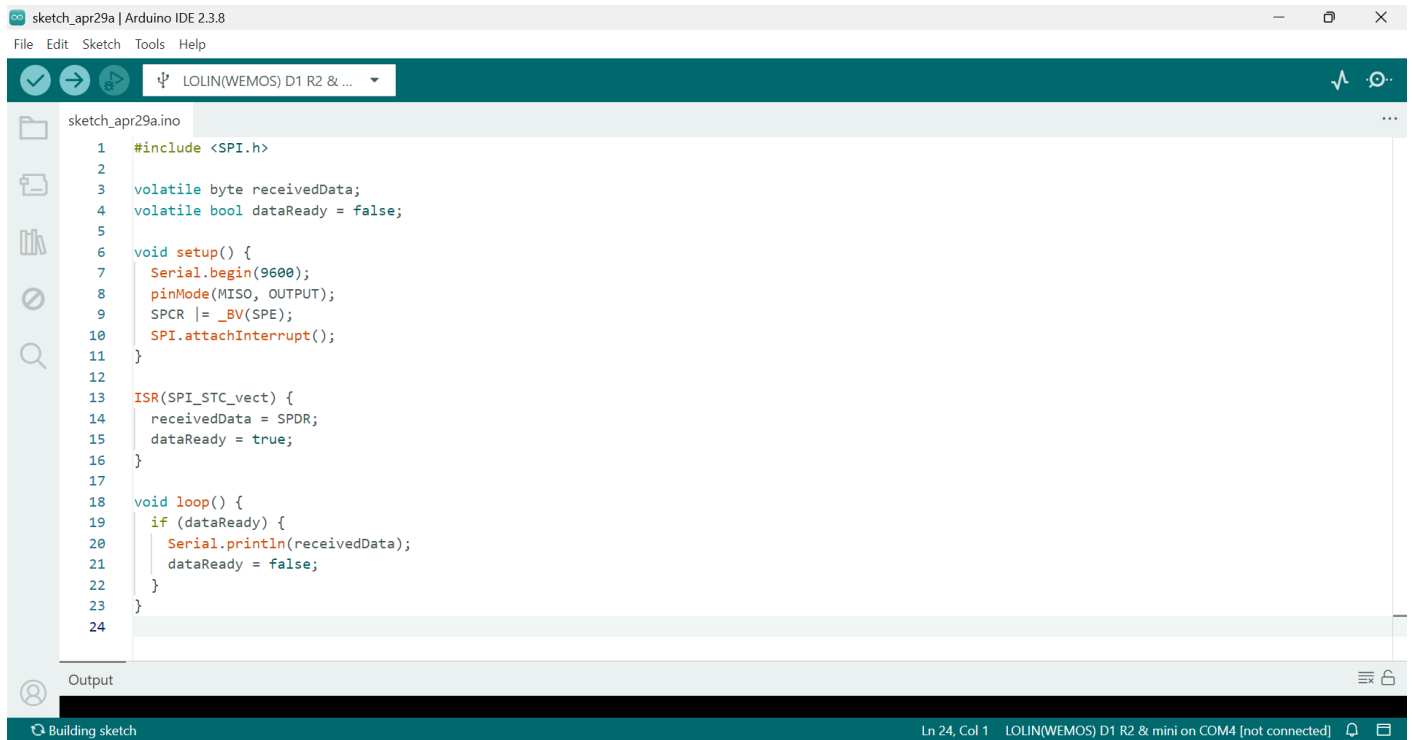
Ln 15, Col 1 LOLIN(WEMOS) D1 R2 & mini on COM4 [not connected]

12:39 26-06-2026

<code>SPISettings(speed, bitOrder, mode)</code>	Sets clock speed, bit order (MSBFIRST/LSBFIRST), and SPI mode (0–3)
<code>SPI.transfer(byte)</code>	Sends one byte and simultaneously receives one byte from slave
<code>CS LOW / HIGH</code>	Must bracket every transaction to select and release the slave

06 ESP32 SPI Slave — Arduino Code

The ESP32 has a dedicated **VSPI** hardware slave peripheral. Use the **ESP32SPISlave** library for full slave support



```
sketch_apr29a | Arduino IDE 2.3.8
File Edit Sketch Tools Help
ψ LOLIN(WEMOS) D1 R2 &...
sketch_apr29a.ino
1  #include <SPI.h>
2
3  volatile byte receivedData;
4  volatile bool dataReady = false;
5
6  void setup() {
7      Serial.begin(9600);
8      pinMode(MISO, OUTPUT);
9      SPCR |= _BV(SPE);
10     SPI.attachInterrupt();
11 }
12
13 ISR(SPI_STC_vect) {
14     receivedData = SPDR;
15     dataReady = true;
16 }
17
18 void loop() {
19     if (dataReady) {
20         Serial.println(receivedData);
21         dataReady = false;
22     }
23 }
24
Output
Building sketch
Ln 24, Col 1  LOLIN(WEMOS) D1 R2 & mini on COM4 [not connected]
```

07 SPI.h — Available Methods Reference

<code>SPI.begin(sck, miso, mosi, ss)</code>	Initialises the SPI bus. On ESP32 you can pass custom pin numbers. Call in <code>setup()</code> before any transfer.
<code>SPI.end()</code>	Releases the SPI bus and all associated pins. Call when done.
<code>SPI.beginTransaction(SPISettings(clk, order, mode))</code>	Locks the bus for exclusive use and applies settings. clk =Hz, order =MSBFIRST/LSBFIRST, mode =0–3.
<code>SPI.endTransaction()</code>	Releases the bus lock. Always pair with <code>beginTransaction()</code> .

<code>SPI.transfer(byte)</code>	Sends 1 byte and simultaneously returns the byte received from slave. Core transfer function.
<code>SPI.transfer16(word)</code>	Sends a 16-bit word (2 bytes) and returns a 16-bit response. Faster than two <code>transfer()</code> calls.
<code>SPI.transfer(buffer, size)</code>	Sends and receives <i>size</i> bytes in-place (buffer is overwritten with received data). Ideal for burst transfers.
<code>SPI.setClockDivider(divider)</code>	Legacy method — sets SPI clock as a division of the system clock. Prefer <code>SPISettings</code> for new code.
<code>SPI.setDataMode(mode)</code>	Sets SPI mode (0–3) determining clock polarity (CPOL) and phase (CPHA). Mode 0 is most common.
<code>SPI.setBitOrder(order)</code>	MSBFIRST (default, most common) or LSBFIRST. Must match the slave device datasheet.
<code>SPI.usingInterrupt(irq)</code>	Tells the SPI library that the given interrupt also uses SPI, so <code>beginTransaction()</code> can disable it safely.
<code>Serial.available()</code>	Not part of SPI.h — this is a UART method. Returns bytes waiting in the serial receive buffer. Used for debug logging alongside SPI.

08 SPI Modes — CPOL & CPHA

Mode	CPOL	CPHA	Clock Idle	Data Sampled On	Common Use
SPI_MODE0	0	0	LOW	Rising Edge	Most sensors, SD cards
SPI_MODE1	0	1	LOW	Falling Edge	Some ADCs
SPI_MODE2	1	0	HIGH	Falling Edge	Some displays
SPI_MODE3	1	1	HIGH	Rising Edge	Some flash chips

Tip: Check your peripheral's datasheet for the correct mode. Using the wrong mode = garbled data. MODE0 works for 90% of common sensors.

09 ESP32 Has 4 SPI Buses

Bus	Name	Purpose	SCLK	MISO	MOSI	CS
SPI0	SPI0	Internal flash (do not use)	—	—	—	—
SPI1	SPI1	Internal flash (do not use)	—	—	—	—
SPI2	HSPI	General purpose ■	14	12	13	15
SPI3	VSPI	General purpose ■	18	19	23	5

■ What to Include When You Comment 'SPI'

When you share this PDF or ask followers to comment **"SPI"**, make sure your post covers these points so it provides real value:

- **What SPI is** — synchronous full-duplex serial protocol
- **The 4 pins** — MOSI, MISO, SCLK, CS and what each does
- **Master/Slave model** — how ESP32 acts as master
- **Arduino code** — both `SPI.begin()` and `SPI.transfer()` examples
- **SPI.h method list** — quick reference for their projects